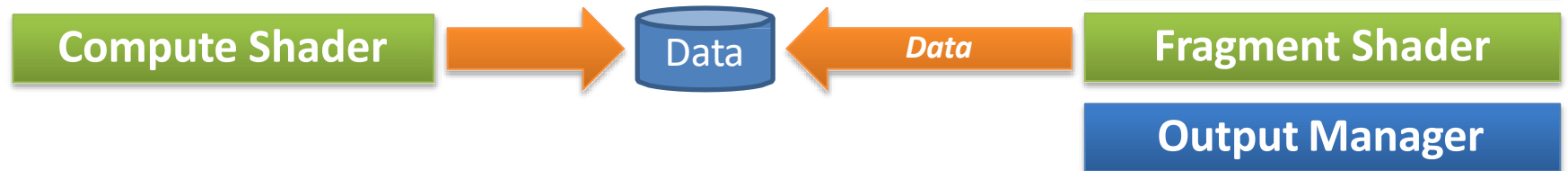


Geometry shader et compute shader

HAI9111

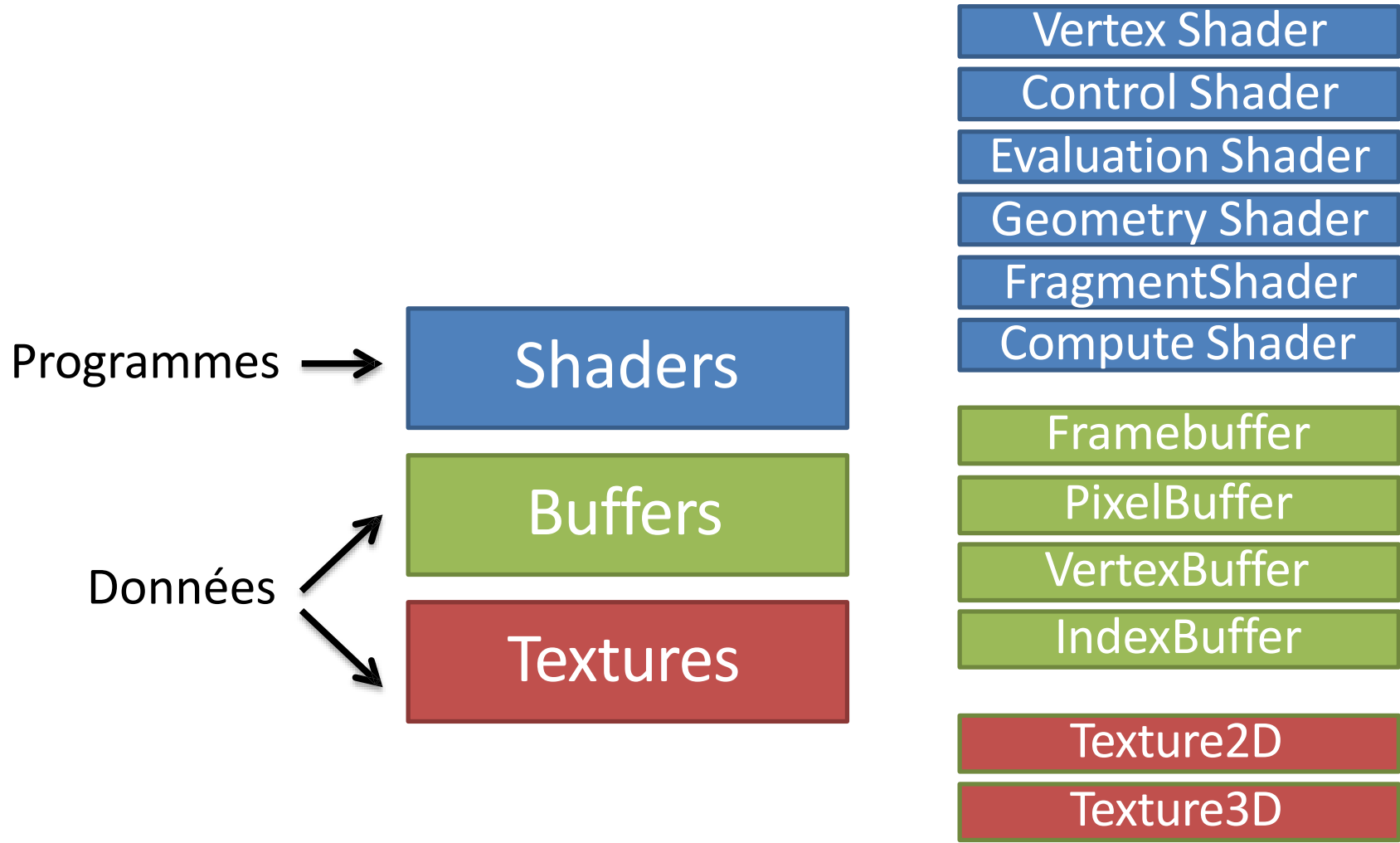
Pipeline Graphique Moderne

- Direct3D 11+ / OpenGL 4+
- Vue API
- Collaboration GPU computing possible (CUDA/OpenCL)
- Implémentation sur processeurs de flux génériques
- Compute Shaders : calcul non graphique de support (mini GPU Computing)



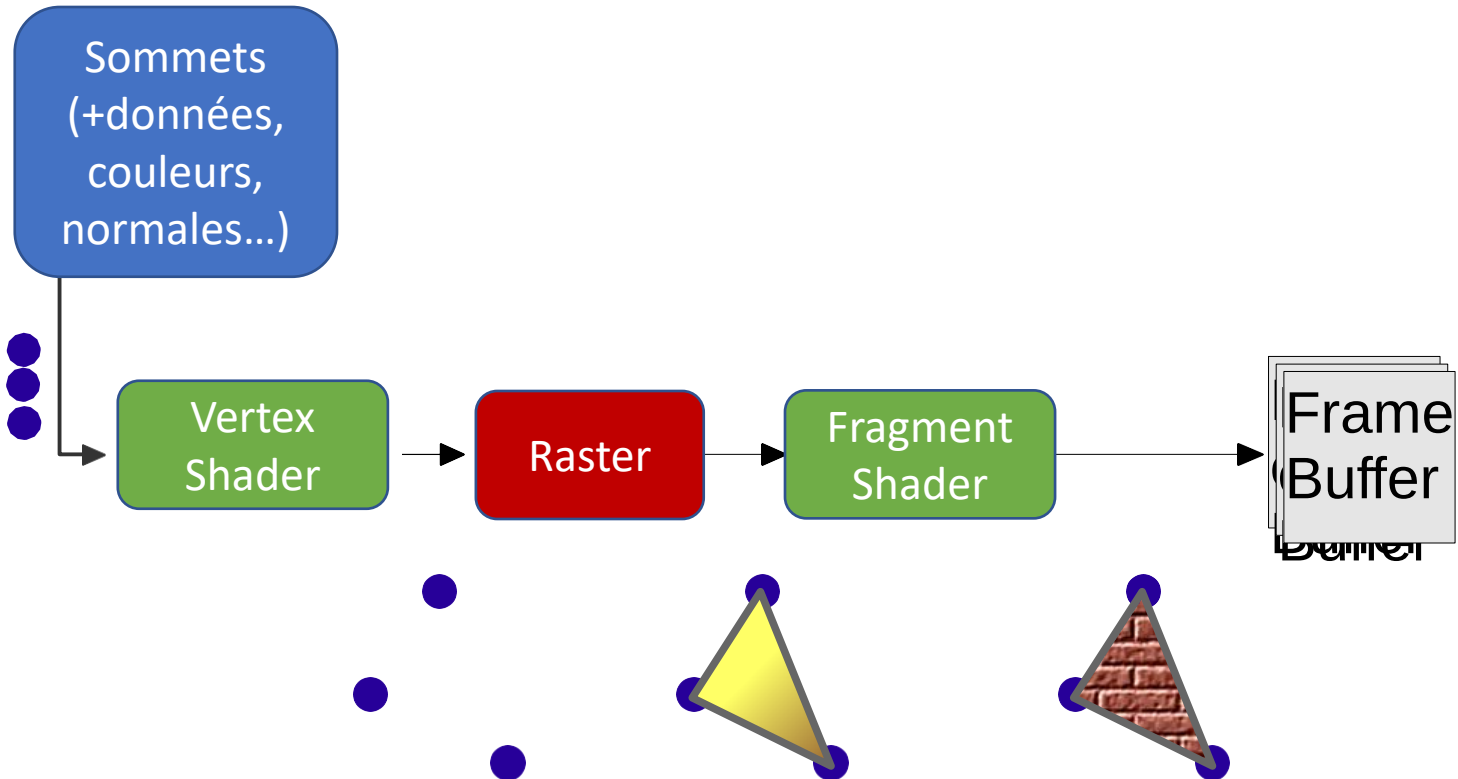
 Programmable  Configurable  Fixe

OpenGL Moderne (v4.x)

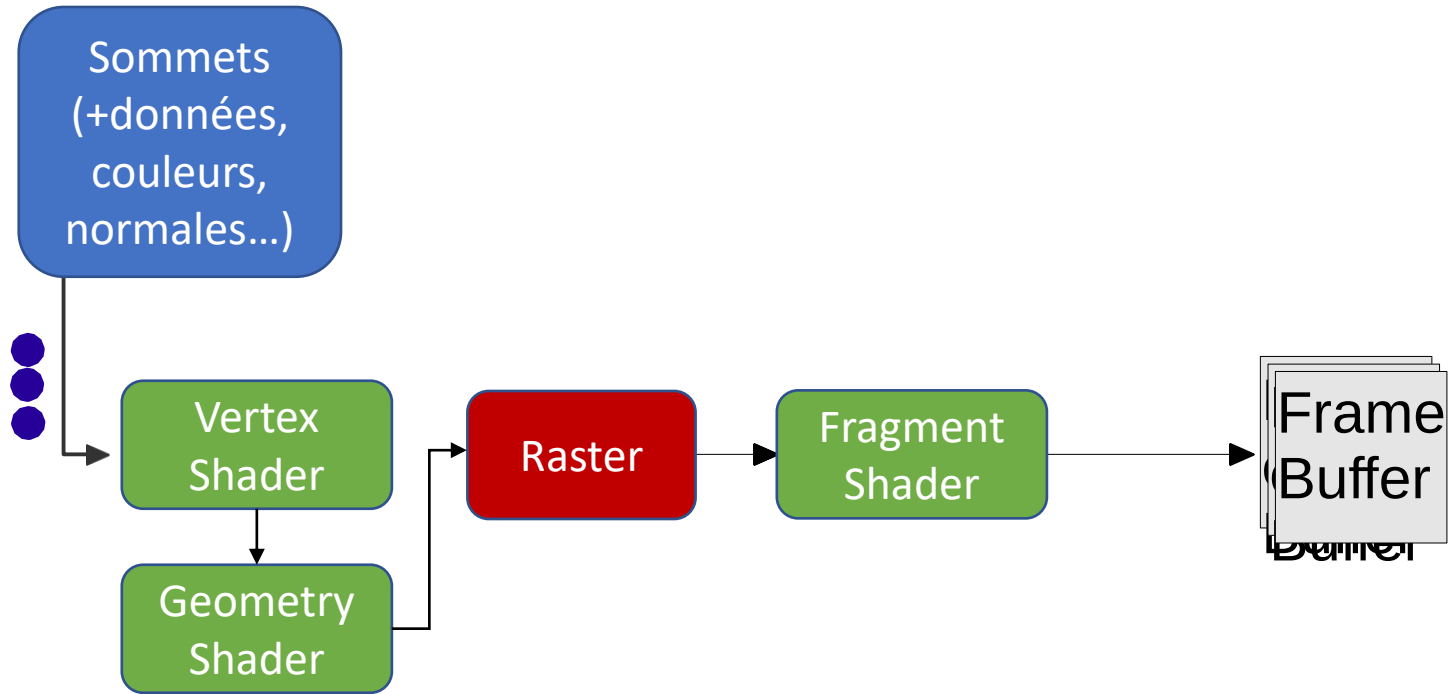


Geometry Shaders

Pipeline Graphique sur GPU



Pipeline Graphique sur GPU



Pipeline Graphique sur GPU

- Le vertex shader traite les sommets indépendamment



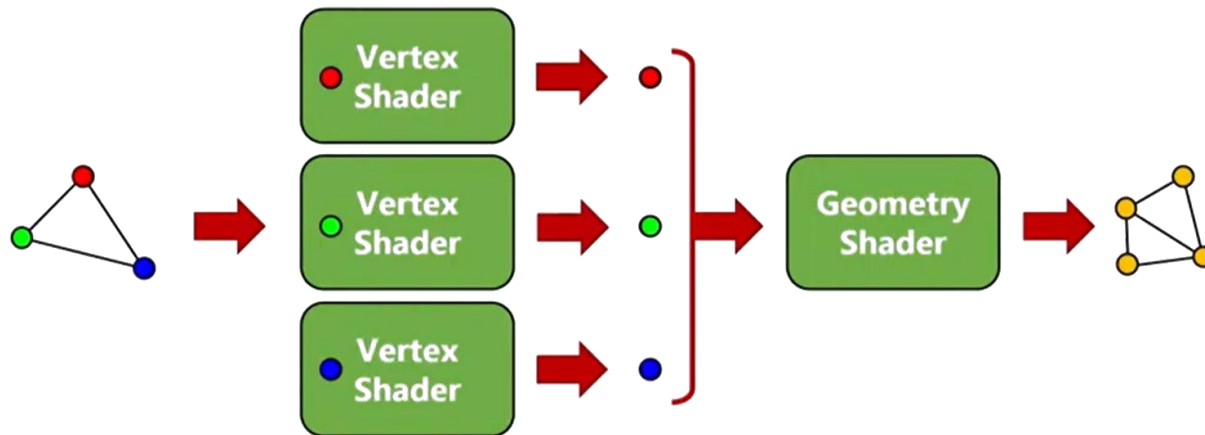
- Le geometry shader :
 - Reçoit tous les sommets de la primitive
 - Sortie libre (ici 4 sommets)
 - 2 triangles
 - envoyée au rasterizer

Pipeline Graphique sur GPU

- Pourquoi ne pas envoyer directement la primitive ?

→ Plus de parallélisme

- Au niveau des sommets pour le vertex shader
- Au niveau des primitives pour le geometry shader



- Geometry shader permet d'avoir des informations sur les sommets de la primitive

Geometry shader

- Ajout de primitives adaptée au GS

Input Types:

- Point
- Line
- Triangle
- Line with Adjacency
- Triangle with Adjacency

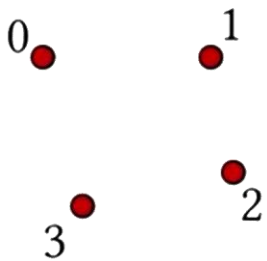


Output Types:

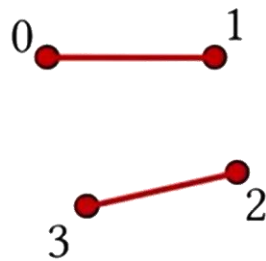
- Points
- Line Strips
- Triangle Strips

Primitives OpenGL

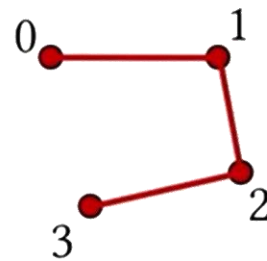
(points)
GL_POINTS



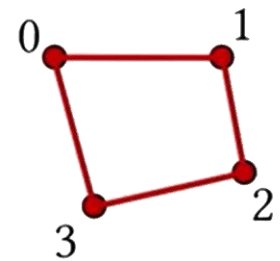
(lines)
GL_LINES



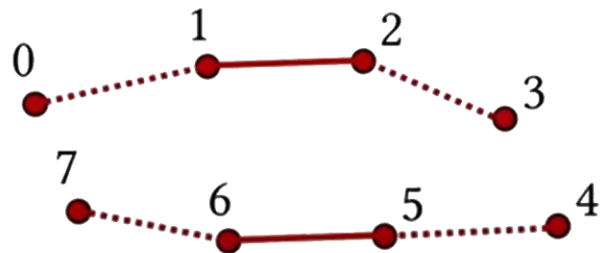
(lines)
GL_LINE_STRIP



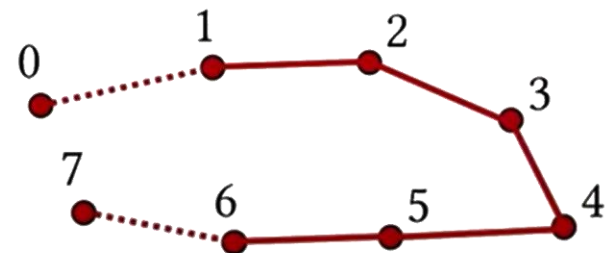
(lines)
GL_LINE_LOOP



(lines_adjacency)
GL_LINES_ADJACENCY

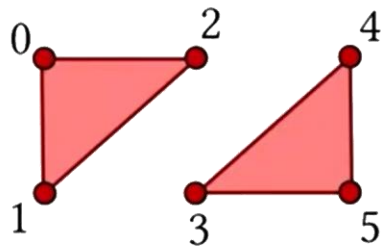


(lines_adjacency)
GL_LINE_STRIP_ADJACENCY

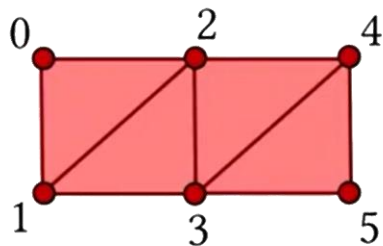


Primitives OpenGL

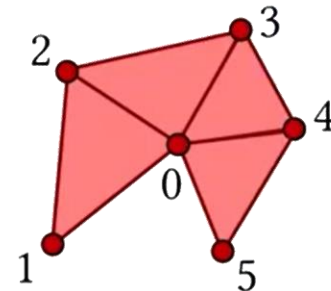
(triangles)
GL_TRIANGLES



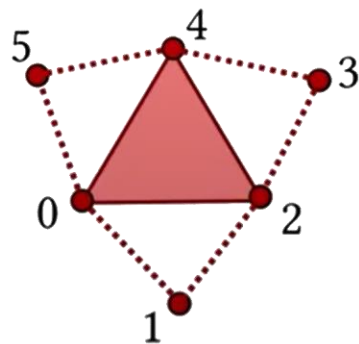
(triangles)
GL_TRIANGLE_STRIP



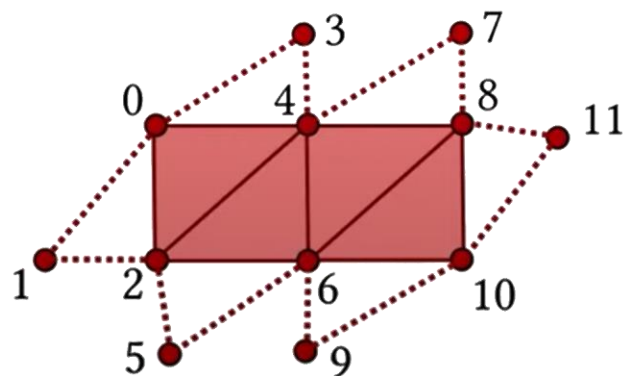
(triangles)
GL_TRIANGLE_FAN



(triangles_adjacency)
GL_TRIANGLES_ADJACENCY



(triangles_adjacency)
GL_TRIANGLE_STRIP_ADJACENCY



Geometry shader

Manipulation d'un primitive :

Triangle : un *geometry shader* (shader de géométrie) a accès aux trois sommets d'un triangle en même temps.

- Les *geometry shaders* permettent de :
 - Sortir le même triangle sans le modifier,
 - Sortir le même triangle avec des modifications,
 - Générer des triangles supplémentaires,
 - Générer un autre type de primitive (comme une ligne),
 - Supprimer le triangle (ne pas le générer du tout).

Geometry shader

- Ajout de primitives adaptée au GS

Input Types:

- points
- lines
- triangles
- lines_adjacency
- triangles_adjacency



Output Types:

- point
- line_strip
- triangle_strip

Geometry Shader

```
#version 330 core

layout ( triangles ) in;
layout ( line_strip, max_vertices = 5 ) out;

void main()
{
    ...
}
```

GLSL

Geometry shader

- Type `gl_in` prédéfinit avec [] → un array de données de sommets

Input Types: Vertex Count

points	1
lines	2
triangles	3
lines_adjacency	4
triangles_adjacency	6

```
in gl_PerVertex
{
    vec4    gl_Position;
    float   gl_PointSize;
    float   gl_ClipDistance[];
} gl_in[];
```

Geometry Shader

```
#version 330 core

layout ( triangles ) in;
layout ( line_strip, max_vertices = 5 ) out;

void main()
{
    ...
}
```

GLSL

Primitives autorisées

<i>Entrées</i>	<i>Primitives compatibles avec celles envoyées par glDrawArrays()</i>	<i>#vertices par invocation</i>
points	GL_POINTS	1
lines	GL_LINES, GL_LINE_STRIP	2
lines_adjacency	GL_LINES_ADJACENCY, GL_LINE_STRIP_ADJACENCY	4
triangles	GL_TRIANGLES, GL_TRIANGLE_STRIP, GL_TRIANGLE_FAN	3
triangles_adjacency	GL_TRIANGLES_ADJACENCY, GL_TRIANGLE_STRIP_ADJACENCY	6

Les primitives de sortie doivent être des ***points***, ***line_strip*** ou ***triangle_strip***

Exemple de GS

- EmitVertex() :
 - Copie la position dans le buffer de sortie

Geometry Shader

```
#version 330 core

layout ( triangles ) in;
layout ( line_strip, max_vertices = 5 ) out;

void main()
{
    gl_Position = gl_in[0].gl_Position;
    EmitVertex();

    gl_Position = gl_in[1].gl_Position;
    EmitVertex();
}
```

GLSL



Example de GS

Geometry Shader

```
#version 330 core

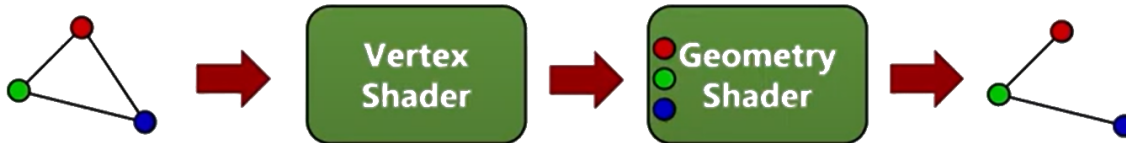
layout ( triangles ) in;
layout ( line_strip, max_vertices = 5 ) out;

void main()
{
    gl_Position = gl_in[0].gl_Position;
    EmitVertex();

    gl_Position = gl_in[1].gl_Position;
    EmitVertex();

    gl_Position = gl_in[2].gl_Position;
    EmitVertex();
}
```

GLSL



Example de GS

Geometry Shader

```
#version 330 core

layout ( triangles ) in;
layout ( line_strip, max_vertices = 5 ) out;

void main()
{
    gl_Position = gl_in[0].gl_Position;
    EmitVertex();
    gl_Position = gl_in[1].gl_Position;
    EmitVertex();
    gl_Position = gl_in[2].gl_Position;
    EmitVertex();
    gl_Position = ( gl_in[0].gl_Position + gl_in[2].gl_Position ) / 2;
    EmitVertex();
}
```

GLSL



Example de GS

- EndPrimitive() appelé automatiquement à la fin de l'exécution du shader

Geometry Shader

```
#version 330 core
layout ( triangles ) in;
layout ( line_strip, max_vertices = 5 ) out;
void main()
{
    gl_Position = gl_in[0].gl_Position;
    EmitVertex();
    gl_Position = gl_in[1].gl_Position;
    EmitVertex();
    EndPrimitive();

    gl_Position = gl_in[2].gl_Position;
    EmitVertex();
    gl_Position = ( gl_in[0].gl_Position + gl_in[2].gl_Position ) / 2;
    EmitVertex();
}
```

GLSL



Exemple de GS

- Données personnalisées en entrée et sortie

Geometry Shader

```
#version 330 core

layout ( triangles ) in;
layout ( line_strip, max_vertices = 5 ) out;

in vec2 someData[];

out vec2 vData;

void main()
{
    gl_Position = gl_in[0].gl_Position;
    vData = someData[0] + someData[1] + someData[2];
    EmitVertex();
    ...
}
```

GLSL



Exemple de GS

- Données personnalisées en entrée et sortie

Geometry Shader

```
#version 330 core

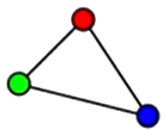
layout ( triangles ) in;
layout ( line_strip, max_vertices = 5 ) out;

in vec2 someData[];

out vec2 vData;

void main()
{
    gl_Position = gl_in[0].gl_Position;
    vData = someData[0] + someData[1] + someData[2];
    EmitVertex();
    ...
}
```

GLSL



Vertex
Shader



Rasterizer

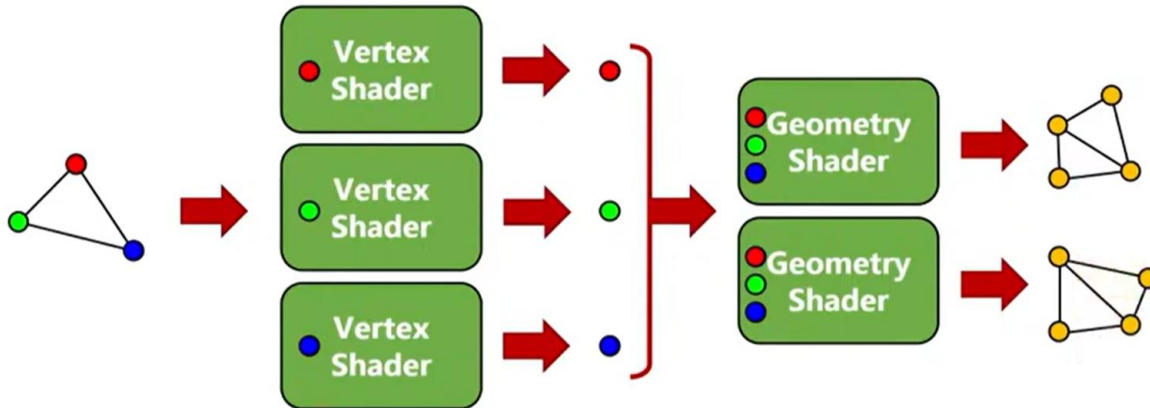


Fragment
Shader

out vec2 someData;

in vec2 vData;

GS instancing



Geometry Shader

```
#version 330 core

layout ( triangles, invocations = 2 ) in;
layout ( line_strip, max_vertices = 5 ) out;

void main()
{
    if ( gl_InvocationID == 0 ) {
        ...
    } else {
        ...
    }
}
```

GLSL

GS instancing

- Avantage :
 - Instances sont exécutées en parallèle

Geometry Shader

```
#version 330 core
```

```
layout ( triangles, invocations = 2 ) in;  
layout ( line_strip, max_vertices = 5 ) out;
```

```
void main()
```

```
{
```

```
    gl_Position = gl_in[ gl_InvocationID ].gl_Position;
```

```
    EmitVertex();
```

```
    gl_Position = gl_in[ gl_InvocationID + 1 ].gl_Position;
```

```
    EmitVertex();
```

```
}
```

GLSL

GS code

- Compiler et lier : similaire aux autres shaders

```
GLuint gShader = glCreateShader(GL_GEOMETRY_SHADER);
```

- Comme pour la tessellation, les attributs arrivent comme des arrays ::

```
... gl_in[2].gl_Position    // position of the 3rd vertex
```

- Les attributs de sortie sont des scalaires :

```
gl_Position = ...
```


GS code

- *Layout qualifiers* : type des primitives et taille des sorties:

```
layout (triangles) in;
```

```
...
```

```
layout (triangle_strip, max_vertices=3) out;
```

- **EmitVertex()** un sommet est spécifié comme une sortie.
- **EndPrimitive()** fin de la construction d'une primitive.
- Variable build-in **gl_PrimitiveIDIn** un ID entiere de la primitive courante commençant à 0.

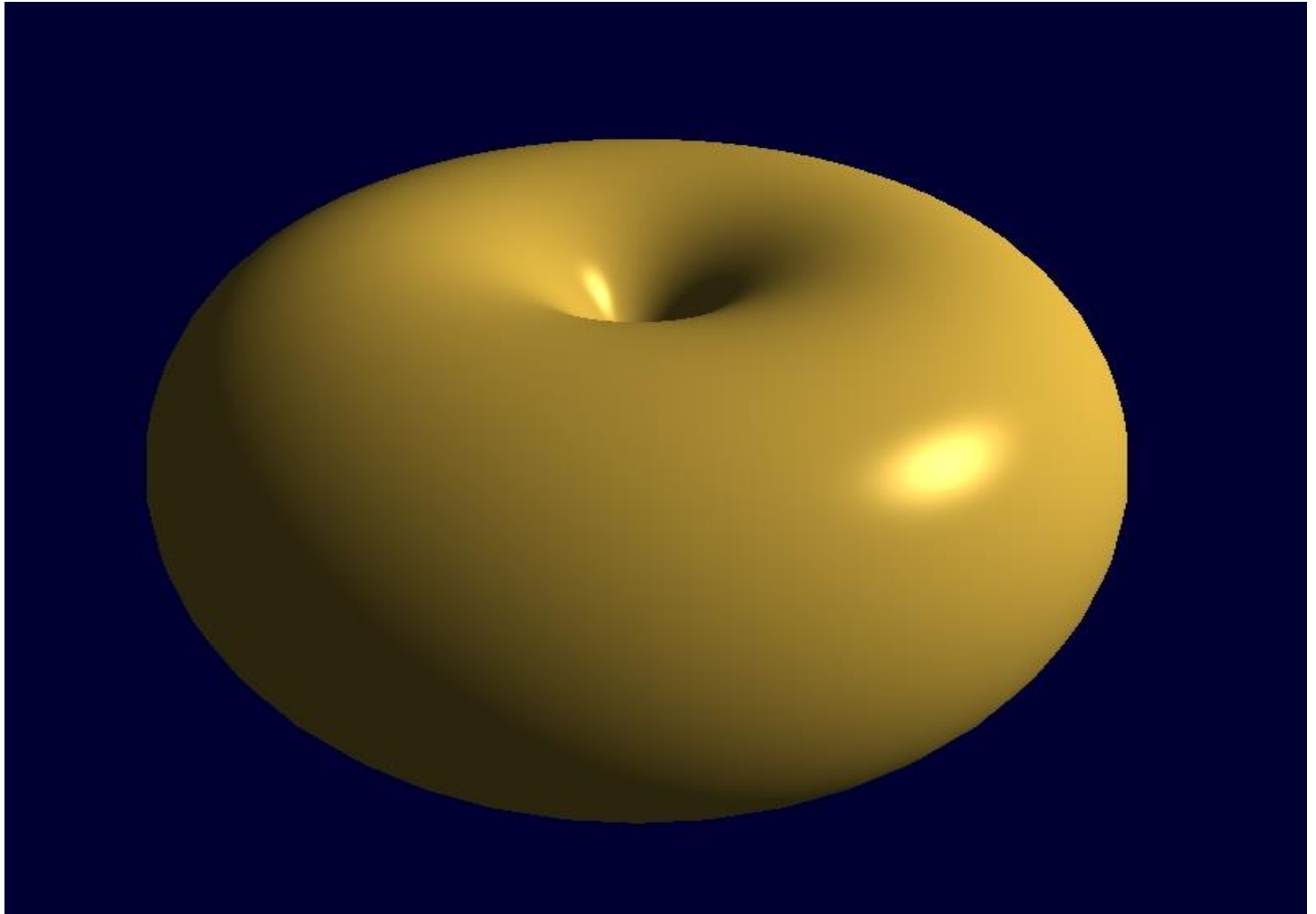
GS example (alteration)

```
#version 430
layout (triangles) in;
in vec3 varyingNormal[ ];    // inputs from the vertex shader (note they are arrays)
in vec3 varyingLightDir[ ];
in vec3 varyingHalfVector[ ];

out vec3 varyingNormalG; // outputs through the rasterizer to the fragment shader
out vec3 varyingLightDirG;           (note they are scalars)
out vec3 varyingHalfVectorG;

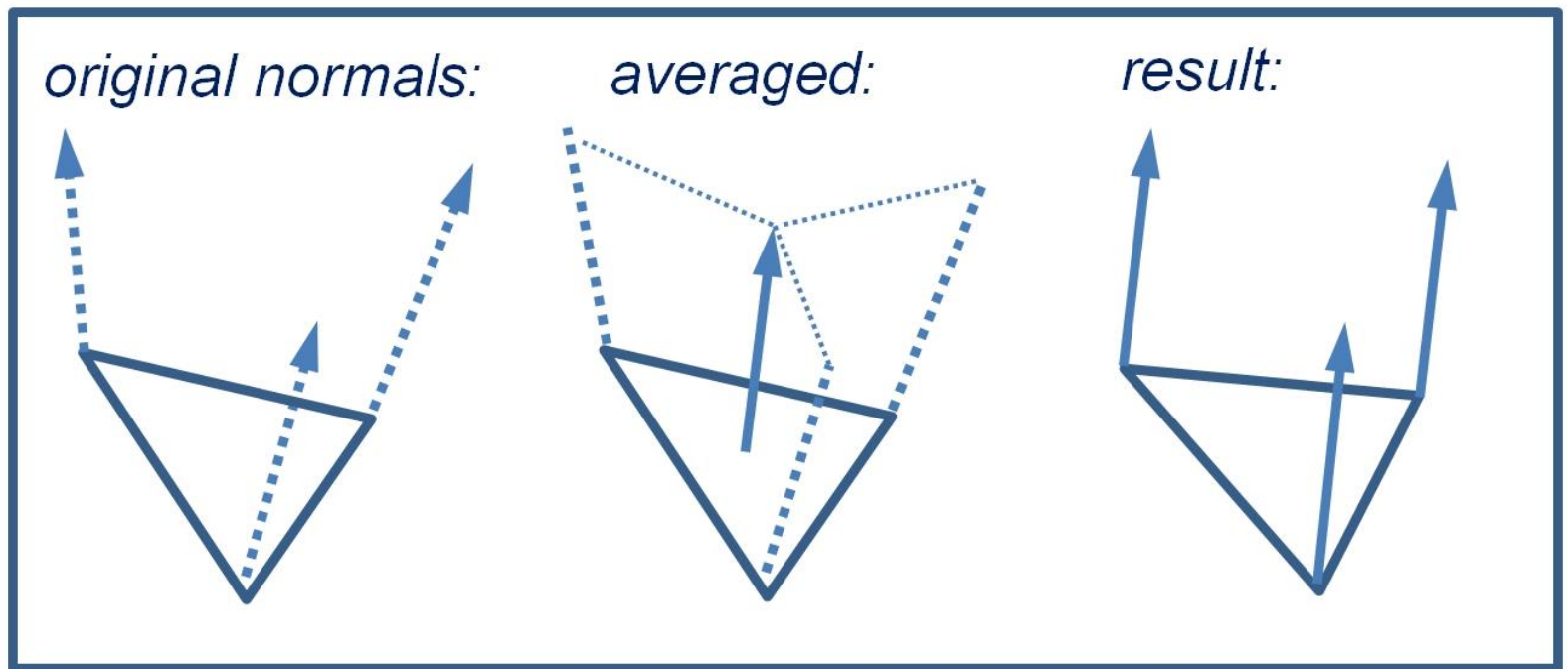
layout (triangle_strip, max_vertices=3) out;
...
void main (void)    // move vertices outward along the surface normal
{ for (i=0; i<3; i++)
    { gl_Position = proj_matrix * gl_in[i].gl_Position
      + normalize(vec4(varyingNormal[i],1.0)) * 0.4;
      varyingNormalG = varyingNormal[i];
      varyingLightDirG = varyingLightDir[i];
      varyingHalfVectorG = varyingHalfVector[i];
      EmitVertex();
    }
    EndPrimitive();
}
```

Tore « gonflé »



Altering primitives (another example)

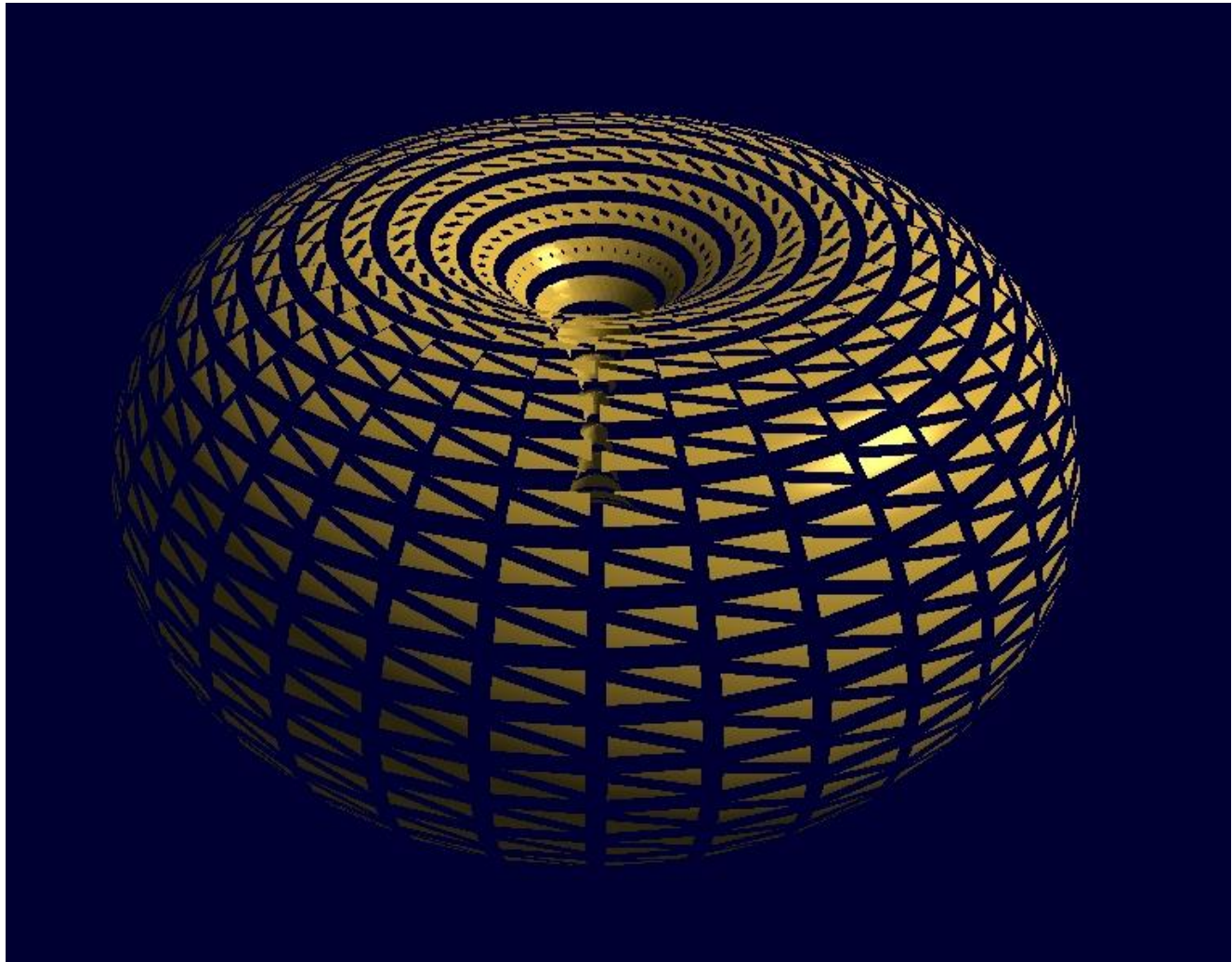
Moving vertices outward along the surface normal averaged for the entire triangle:



Geometry Shader

```
void main (void)
{ // average the three triangle vertex normals, creating a single triangle normal
  vec4 triangleNormal =
    vec4(((varyingNormal[0] + varyingNormal[1] + varyingNormal[2]) / 3.0),1.0);
  // move all three vertices outward along the same normal
  for (i=0; i<3; i++)
  { gl_Position = proj_matrix * (gl_in[i].gl_Position+normalize(triangleNormal)*0.4);
    varyingNormalG = varyingNormal[i];
    varyingLightDirG = varyingLightDir[i];
    varyingHalfVectorG = varyingHalfVector[i];
    EmitVertex();
  }
  EndPrimitive();
}
```

Torus “exploded” by geometry shader:



However, we need to see the inside of the exploded torus...

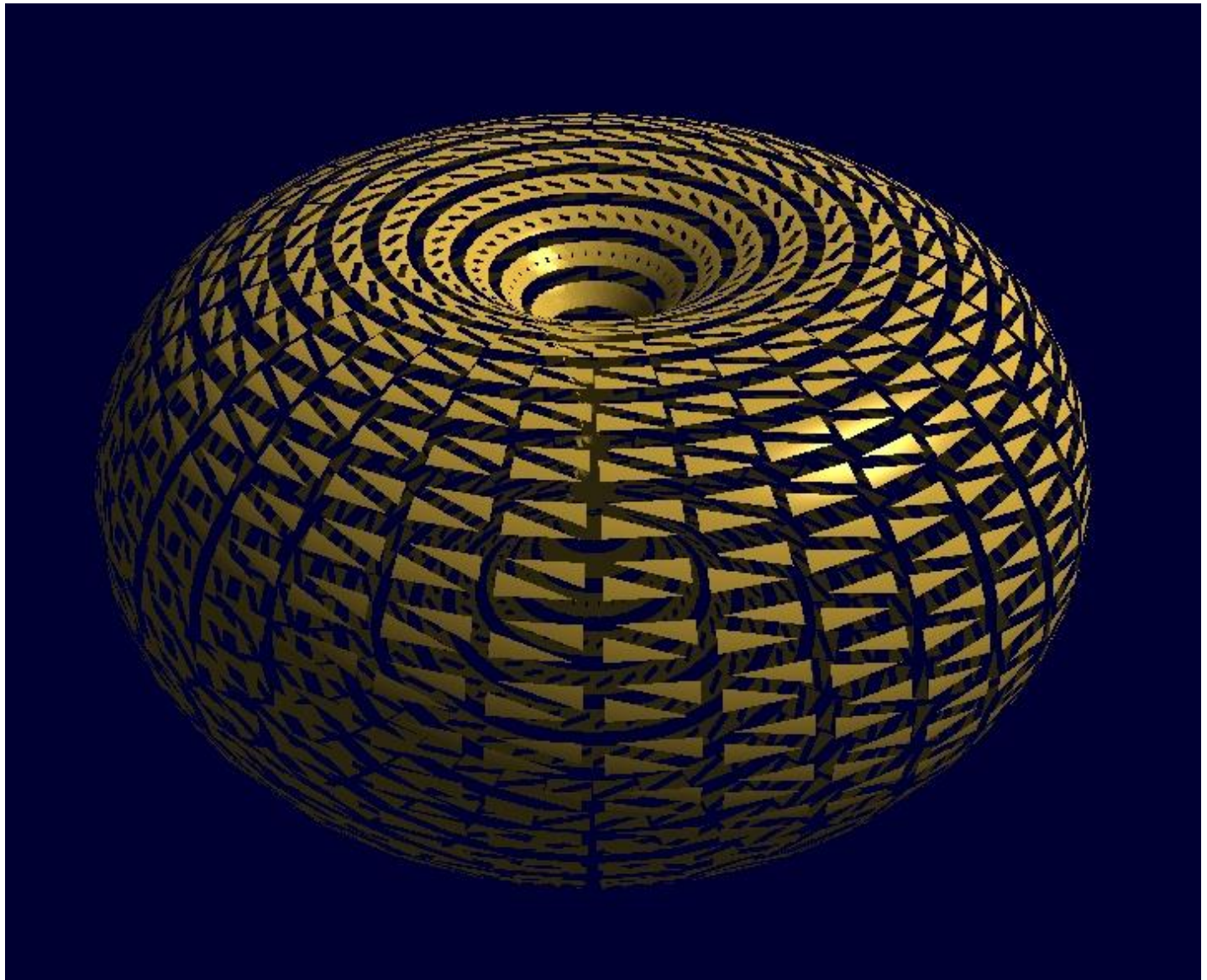
changes to display() function:

```
...  
// draw front-facing triangles – enable lighting  
glUniform1i(enableLightingLoc, 1);  
glFrontFace(GL_CCW);  
glDrawElements(GL_TRIANGLES, numTorusIndices, GL_UNSIGNED_INT, 0);  
// draw back-facing triangles – disable lighting  
glUniform1i(enableLightingLoc, 0);  
glFrontFace(GL_CW);  
glDrawElements(GL_TRIANGLES, numTorusIndices, GL_UNSIGNED_INT, 0);
```

modification to fragment shader:

```
...  
if (enableLighting == 1)  
{ fragColor = ... // when rendering front faces, use normal lighting computations  
}  
else // when rendering back faces, enable only the ambient lighting component  
{ fragColor = globalAmbient * material.ambient + light.ambient * material.ambient;
```


“exploded” torus including back faces:



Deleting primitives (example)

Geometry Shader

// inputs, outputs, and uniforms as before

...

void main (void)

{ if (mod(gl_PrimitiveIDIn,3) != 0)

{ for (int i=0; i<3; i++)

{ gl_Position = proj_matrix * gl_in[i].gl_Position;

varyingNormalG = varyingNormal[i];

varyingLightDirG = varyingLightDir[i];

varyingHalfVectorG = varyingHalfVector[i];

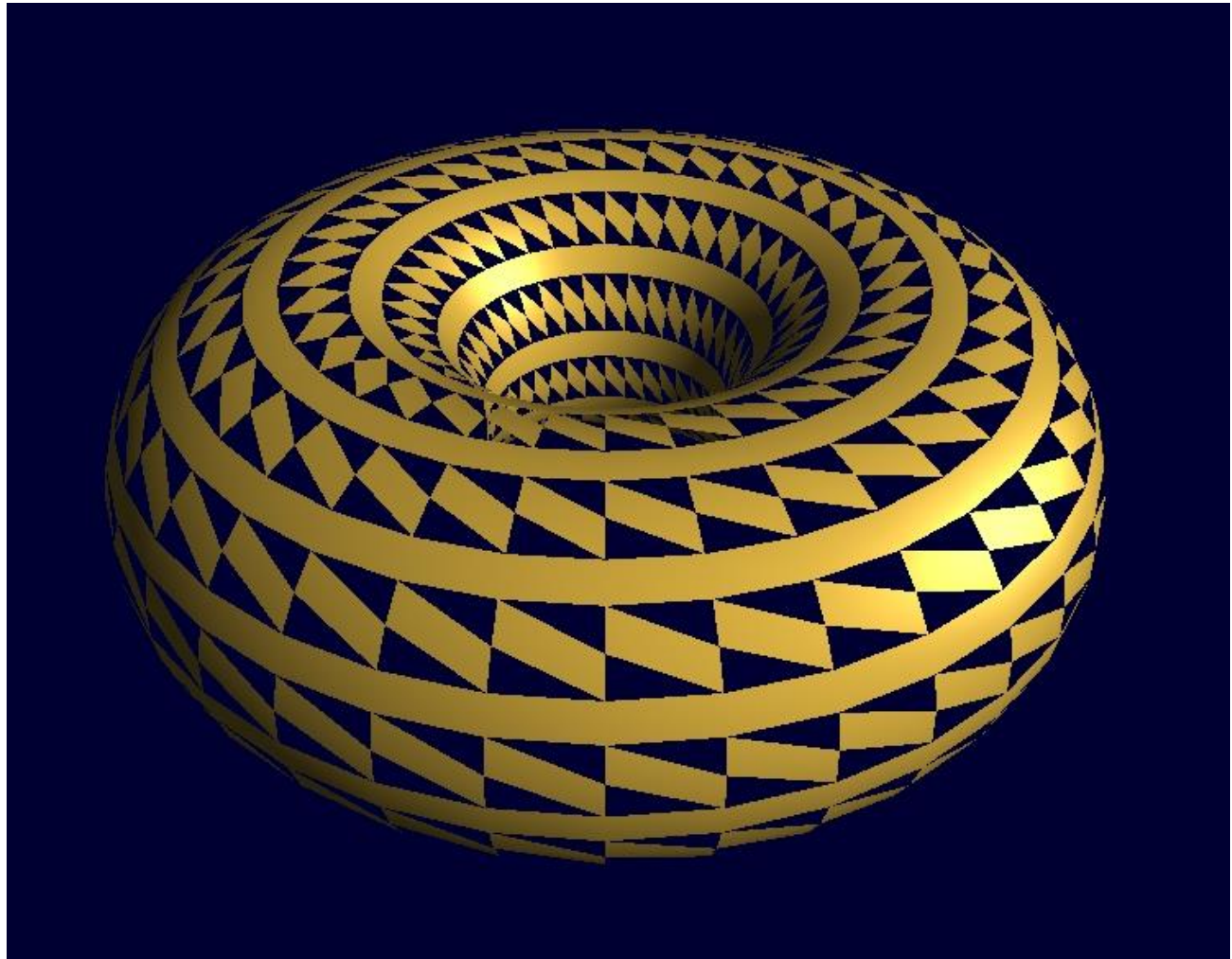
EmitVertex();

} }

EndPrimitive();

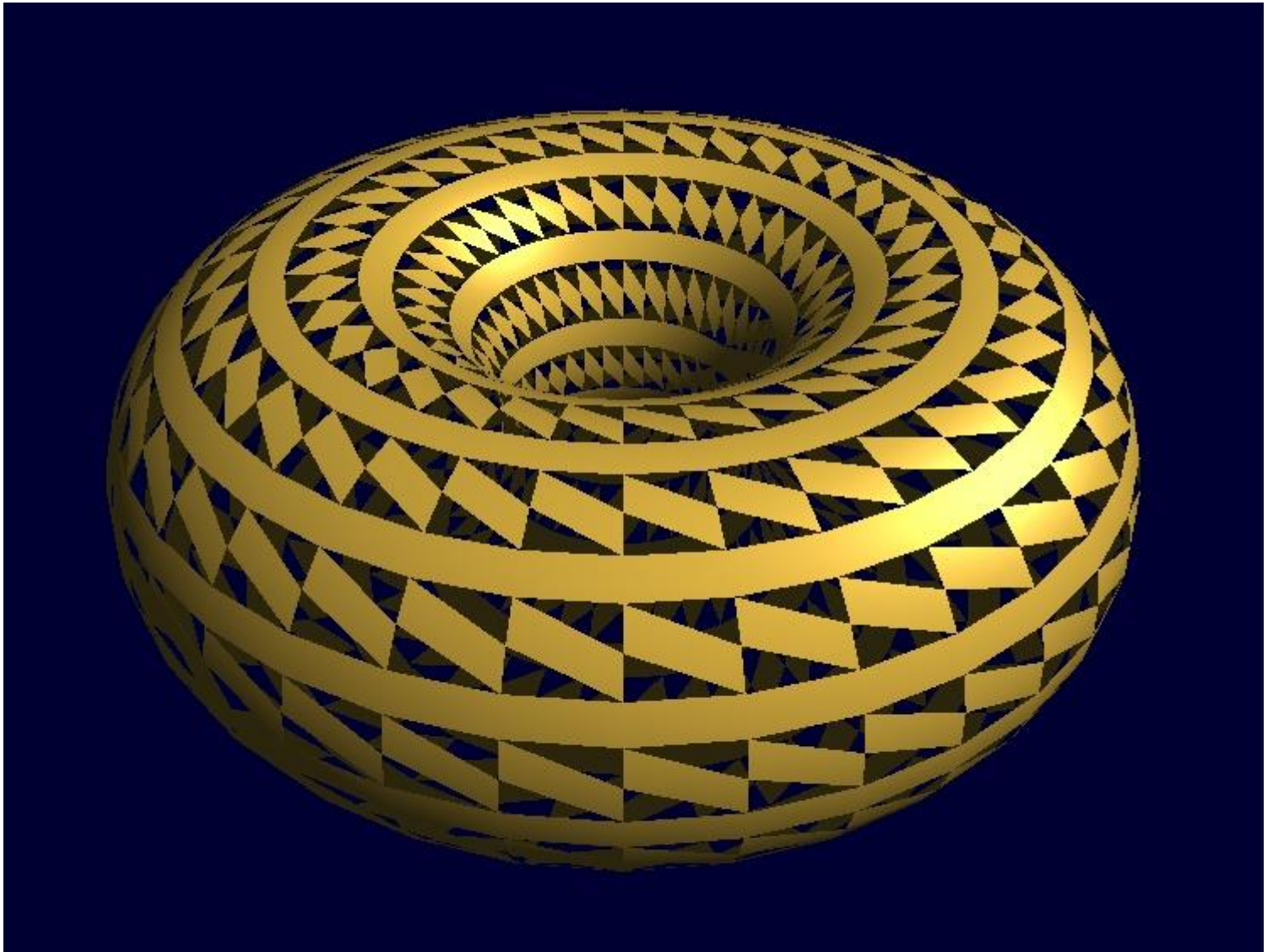
}

Torus with primitives deleted in geometry shader



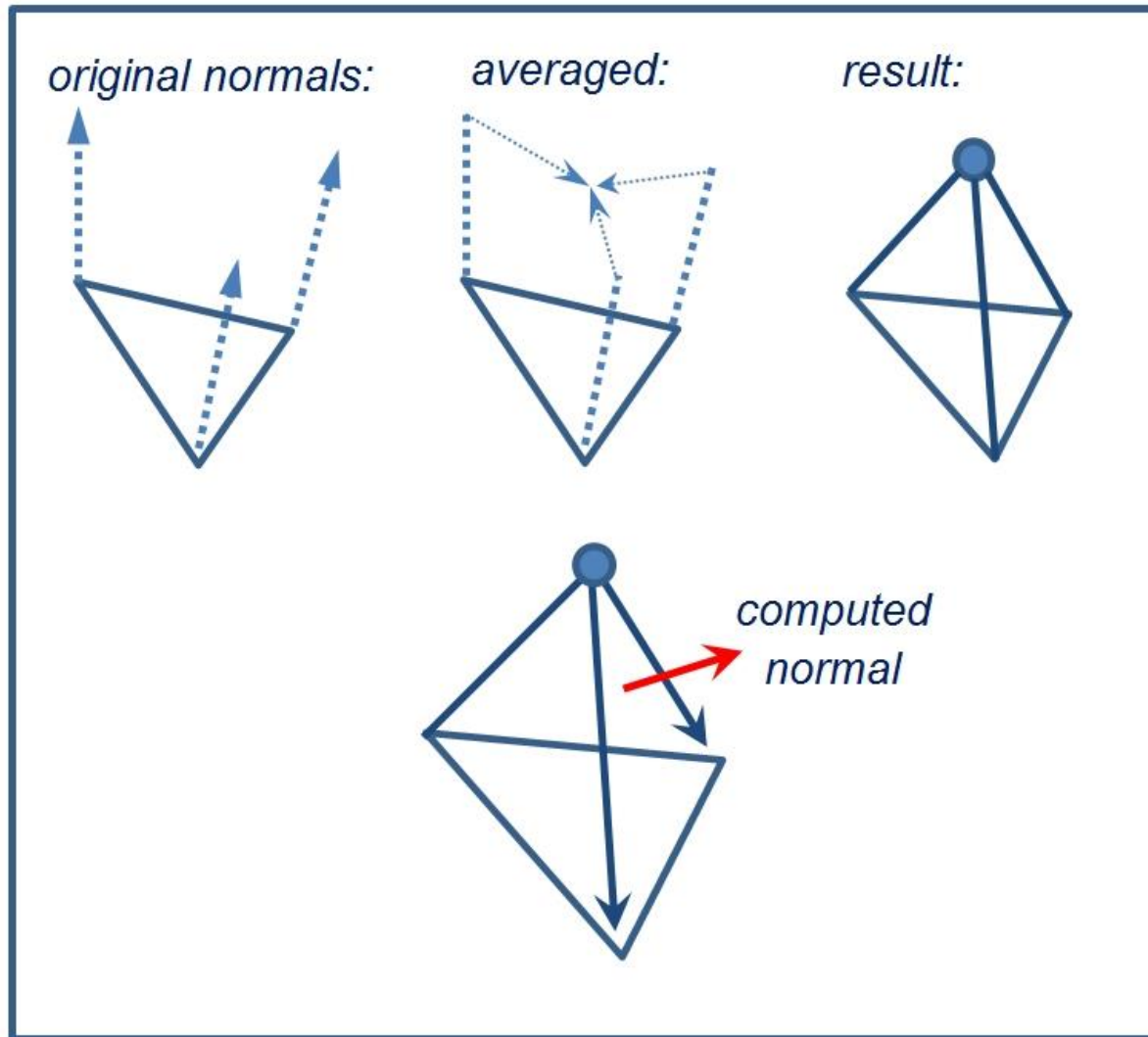
Here too, we need to see the inside of the torus...

Torus with deleted primitives, including back face



Adding primitives

Example: converting triangles to pyramids



Geometry Shader

...

void main (void)

```
{ // offset the three triangle vertices by the original surface normal
    vec3 sp0 = gl_in[0].gl_Position.xyz + varyingOriginalNormal[0]*sLen;
    vec3 sp1 = gl_in[1].gl_Position.xyz + varyingOriginalNormal[1]*sLen;
    vec3 sp2 = gl_in[2].gl_Position.xyz + varyingOriginalNormal[2]*sLen;

    // compute the new points comprising a small pyramid
    newPoints[0] = gl_in[0].gl_Position.xyz;
    newPoints[1] = gl_in[1].gl_Position.xyz;
    newPoints[2] = gl_in[2].gl_Position.xyz;
    newPoints[3] = (sp0+sp1+sp2) / 3.0; // spike point

    // compute the directions from the vertices to the light
    lightDir[0] = light.position – newPoints[0];
    lightDir[1] = light.position – newPoints[1];
    lightDir[2] = light.position – newPoints[2];
    lightDir[3] = light.position – newPoints[3];

    // build three new triangles to form a small pyramid on the surface
    makeNewTriangle(0,1); // the third point is always the spike point
    makeNewTriangle(1,2);
    makeNewTriangle(2,0);

}
```

continued . . .

Geometry Shader (continued)

...

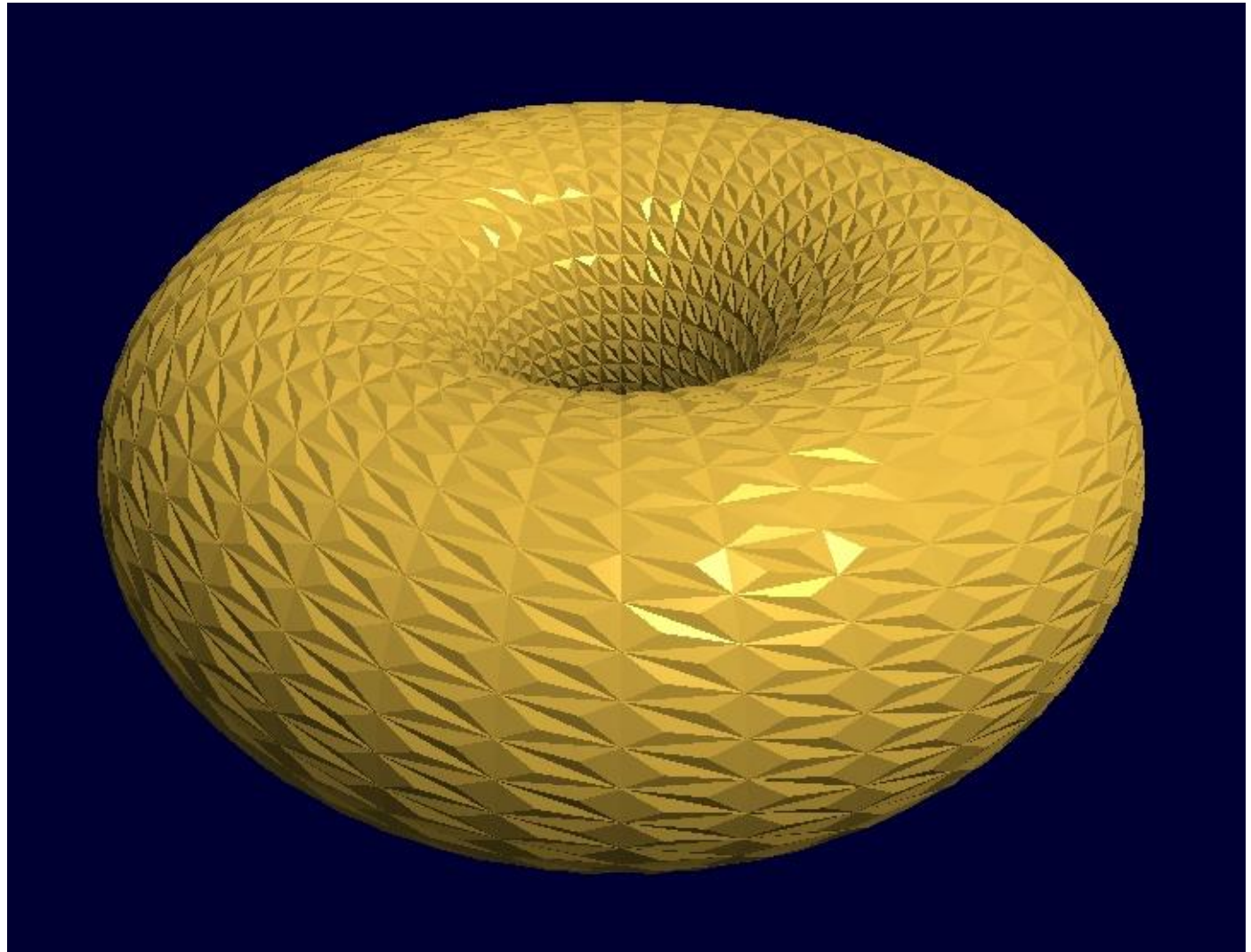
```
vec3 newPoints[ ], lightDir[ ];
float sLen = 0.01;    // sLen is the "spike length", the height of the small pyramid

void setOutputValues(int p, vec3 norm)
{   varyingNormal = norm;
    varyingLightDir = lightDir[p];
    varyingVertPos = newPoints[p];
    gl_Position = proj_matrix * vec4(newPoints[p], 1.0);
}

void makeNewTriangle(int p1, int p2)
{   // generate surface normal for this triangle
    vec3 c1 = normalize(newPoints[p1] - newPoints[3]);
    vec3 c2 = normalize(newPoints[p2] - newPoints[3]);
    vec3 norm = cross(c1,c2);

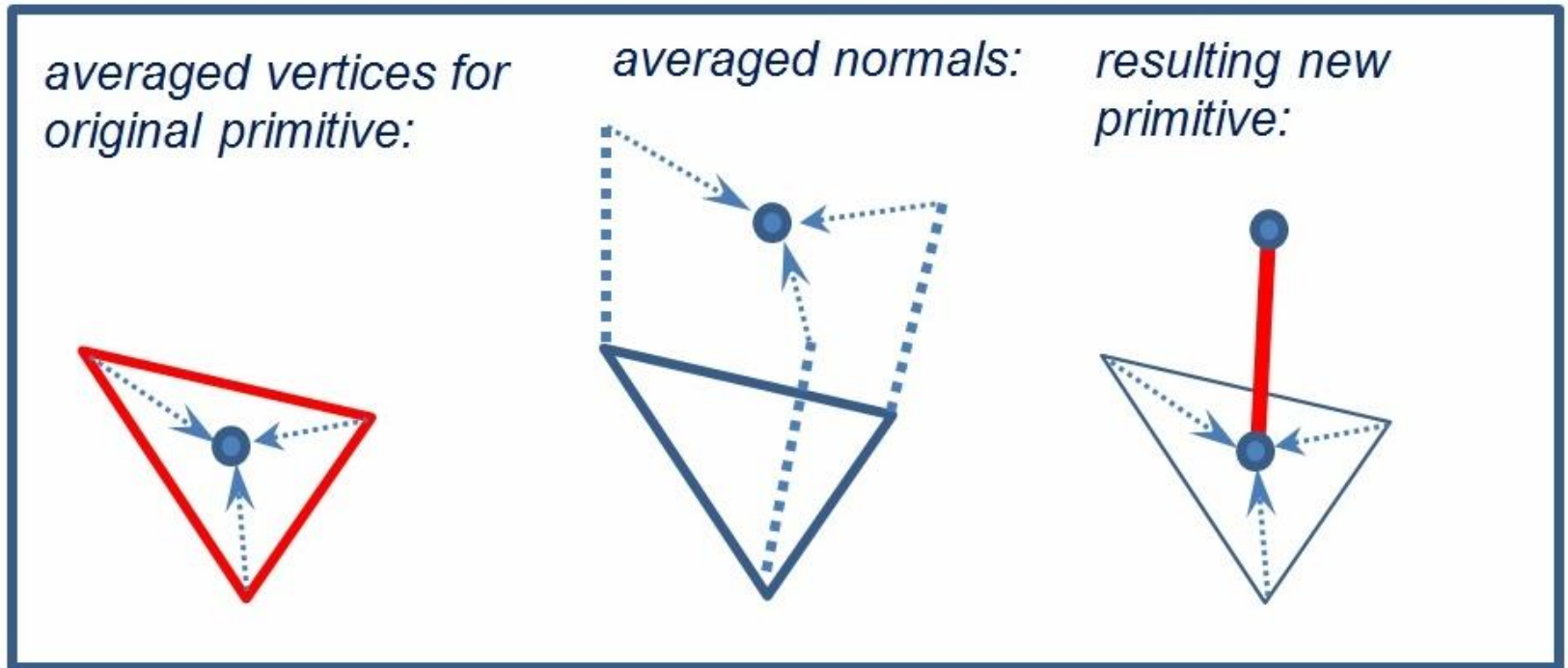
    // generate and emit the three vertices
    setOutputValues(p1, norm); EmitVertex();
    setOutputValues(p2, norm); EmitVertex();
    setOutputValues(3, norm); EmitVertex();
    EndPrimitive();
}
```


*Torus made of small pyramids ...
geometry shader for primitive addition:*



Changing Primitive Types

Example: replacing triangles with lines



Geometry Shader

```
void main(void)
{
    vec3 op0 = gl_in[0].gl_Position.xyz; // original triangle vertices
    vec3 op1 = gl_in[1].gl_Position.xyz;
    vec3 op2 = gl_in[2].gl_Position.xyz;
    vec3 ep0 = gl_in[0].gl_Position.xyz + varyingNormal[0]*sLen; // offset
    vertices
    vec3 ep1 = gl_in[1].gl_Position.xyz + varyingNormal[1]*sLen;
    vec3 ep2 = gl_in[2].gl_Position.xyz + varyingNormal[2]*sLen;

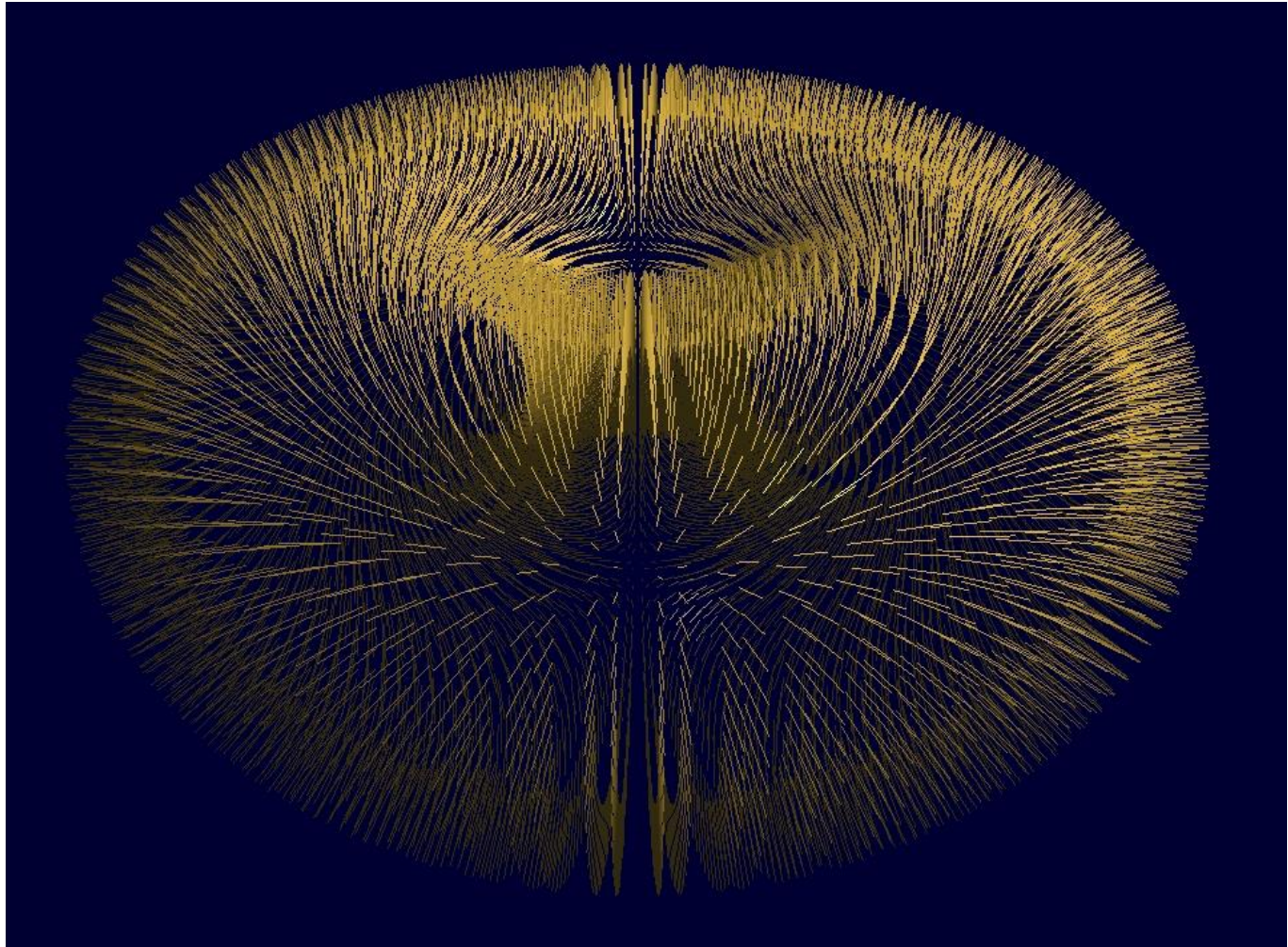
    // compute the new points comprising a small line segment
    vec3 newPoint1 = (op0 + op1 + op2)/3.0;    // original (start) point
    vec3 newPoint2 = (ep0 + ep1 + ep2)/3.0;    // end point
    gl_Position = proj_matrix * vec4(newPoint1, 1.0);

    varyingVertPosG = newPoint1;
    varyingLightDirG = light.position - newPoint1;
    varyingNormalG = varyingNormal[0];
    EmitVertex();

    gl_Position = proj_matrix * vec4(newPoint2, 1.0);
    varyingVertPosG = newPoint2;
    varyingLightDirG = light.position - newPoint2;
    varyingNormalG = varyingNormal[1];
    EmitVertex();
    EndPrimitive();
}
```

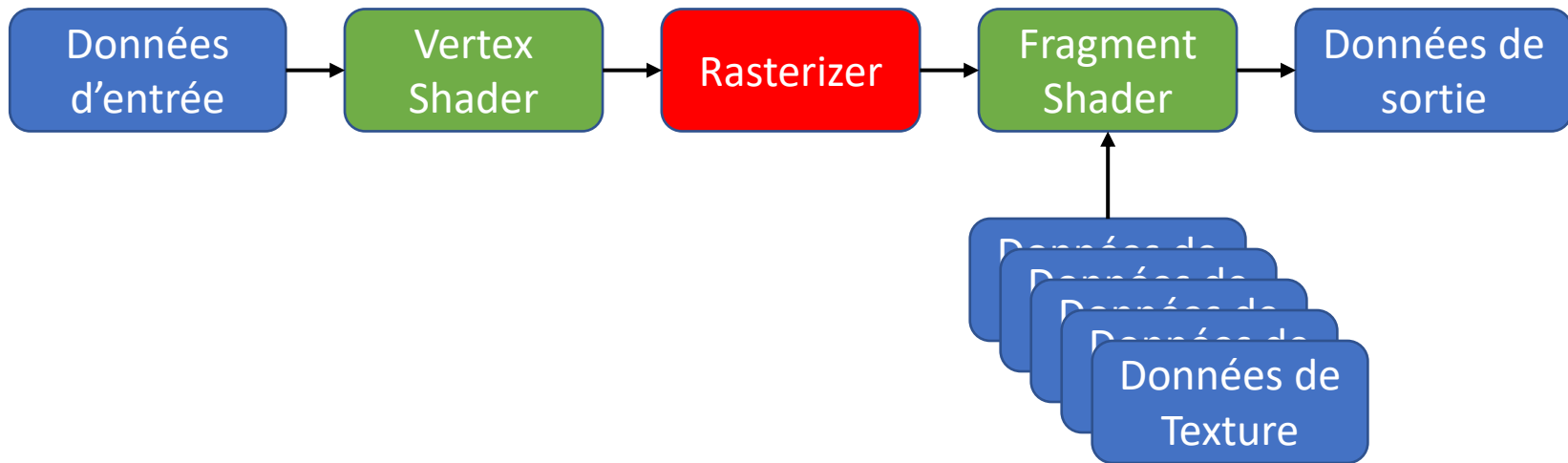
Torus “fur”...

Geometry shader for changing primitive types:

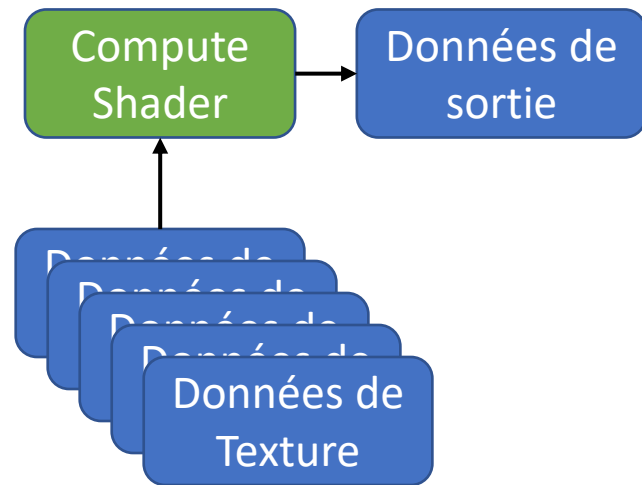


Compute shaders

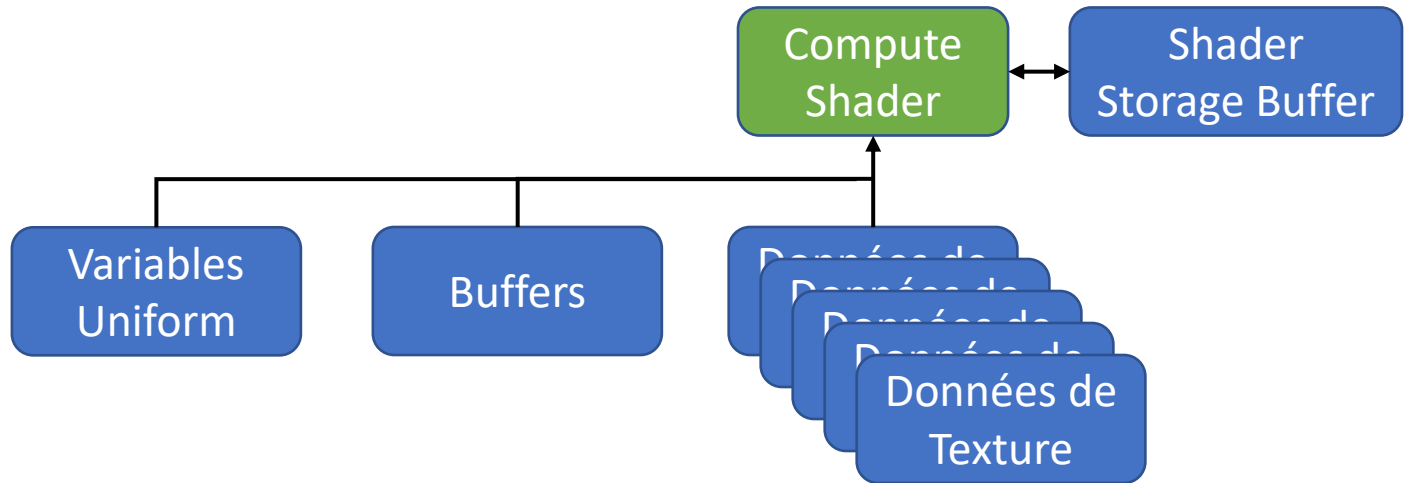
GPGPU



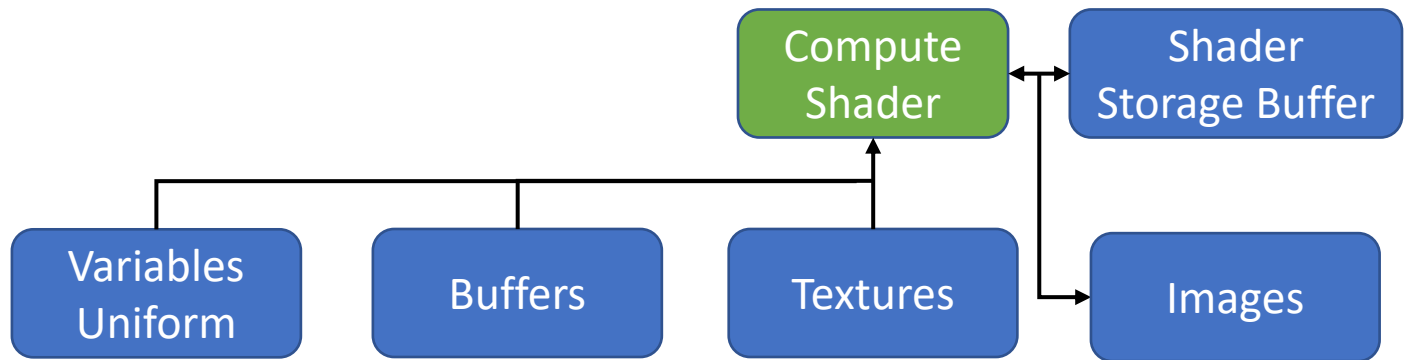
GPGPU



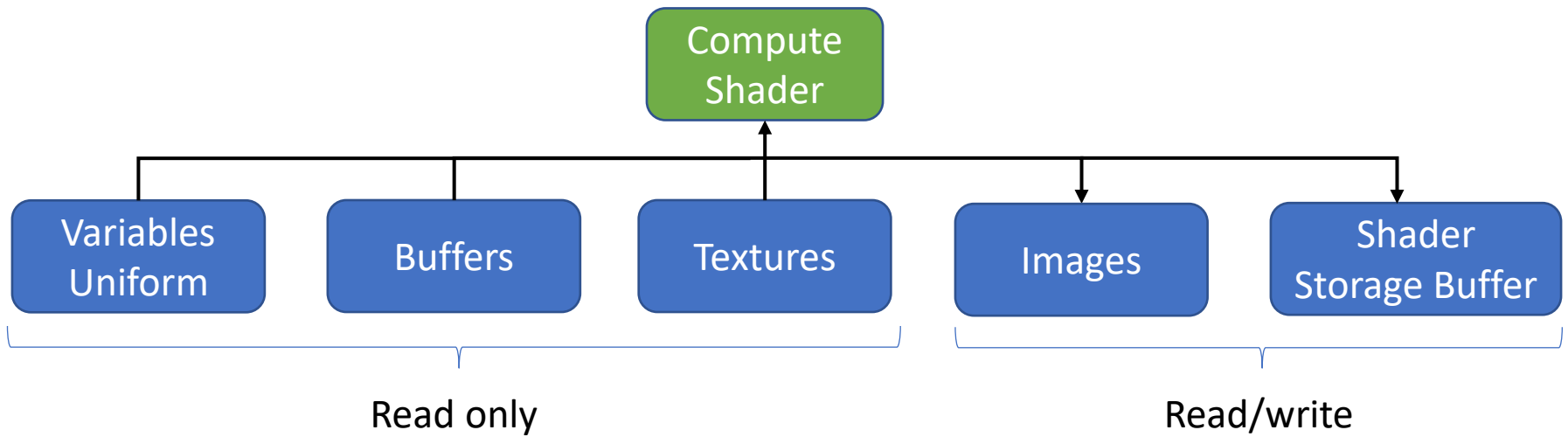
Computer Shader



Compute Shader



Compute Shader



Images GLSL

float → images2D
int → iimages2D
uint → uimages2D

} **g**images2D

gvec4 **imageLoad**(gimage img, IMAGE_COORD);

Integer pixel coordinates

gvec4 **imageStore**(gimage img, IMAGE_COORD, **gvec4** data);

Pas de filtrage (contrairement aux textures)

Images GLSL

float → images2D
int → iimages2D
uint → uimages2D

} **g**images2D

gvec4 **imageLoad**(**gimage** img, **IMAGE_COORD**);

Integer pixel coordinates

gvec4 **imageStore**(**gimage** img, **IMAGE_COORD**, **gvec4** data);

Pas de filtrage (contrairement aux textures)

Images GLSL

Atomic Operations:

```
gint imageAtomicExchange( gimage img, IMAGE_COORDS, gint data );  
gint imageAtomicCompSwap( gimage img, IMAGE_COORDS, gint compare, gint data );  
gint imageAtomicAdd      ( gimage img, IMAGE_COORDS, gint data );  
gint imageAtomicAnd      ( gimage img, IMAGE_COORDS, gint data );  
gint imageAtomicOr       ( gimage img, IMAGE_COORDS, gint data );  
gint imageAtomicXor      ( gimage img, IMAGE_COORDS, gint data );  
gint imageAtomicMin      ( gimage img, IMAGE_COORDS, gint data );  
gint imageAtomicMax      ( gimage img, IMAGE_COORDS, gint data );
```

Images

GLSL

GLSL



<i>gimage1D</i>	GL_TEXTURE_1D
<i>gimage2D</i>	GL_TEXTURE_2D
<i>gimage3D</i>	GL_TEXTURE_3D
<i>gimageCube</i>	GL_TEXTURE_CUBE_MAP
<i>gimage2DRect</i>	GL_TEXTURE_RECTANGLE
<i>gimage1DArray</i>	GL_TEXTURE_1D_ARRAY
<i>gimage2DArray</i>	GL_TEXTURE_2D_ARRAY
<i>gimageCubeArray</i>	GL_TEXTURE_CUBE_MAP_ARRAY
<i>gimageBuffer</i>	GL_TEXTURE_BUFFER
<i>gimage2DMS</i>	GL_TEXTURE_2D_MULTISAMPLE
<i>gimage2DMSArray</i>	GL_TEXTURE_2D_MULTISAMPLE_ARRAY

Images



```
void glBindImageTexture(  
    GLuint    unit,  
    GLuint    texture,  
    GLint     level,  
    GLboolean layered,  
    GLint     layer,  
    GLenum    access, { GL_READ_ONLY  
    GLenum    format  { GL_WRITE_ONLY  
                      { GL_READ_WRITE  
    );
```

Shader Buffer Storage Object

```
int data[SOME_SIZE];
```

```
...
```

```
GLuint ssbo;
```

```
glGenBuffers(1, &ssbo);
```

```
glBindBuffer(GL_SHADER_STORAGE_BUFFER, ssbo);
```

```
glBufferData(GL_SHADER_STORAGE_BUFFER, sizeof(data), data, GLenum usage);
```

```
//sizeof(data) only works for statically sized C/C++ arrays.
```

```
glBindBufferBase(GL_SHADER_STORAGE_BUFFER, 3, ssbo);
```

```
glBindBuffer(GL_SHADER_STORAGE_BUFFER, 0); // unbind
```

https://wikis.khronos.org/opengl/Shader_Storage_Buffer_Object

Shader Buffer Storage Object

GLSL

```
layout( std430, binding = 3 ) buffer layoutName  
{  
    int data_SSBO[];  
} myData;
```

```
myData.data_SSBO[132] = 5;
```

Compiler le compute shader

```
GLuint cs = glCreateShader( GL_COMPUTE_SHADER )  
glShaderSource( cs, 1, &source, NULL );  
glCompileShader( cs );
```

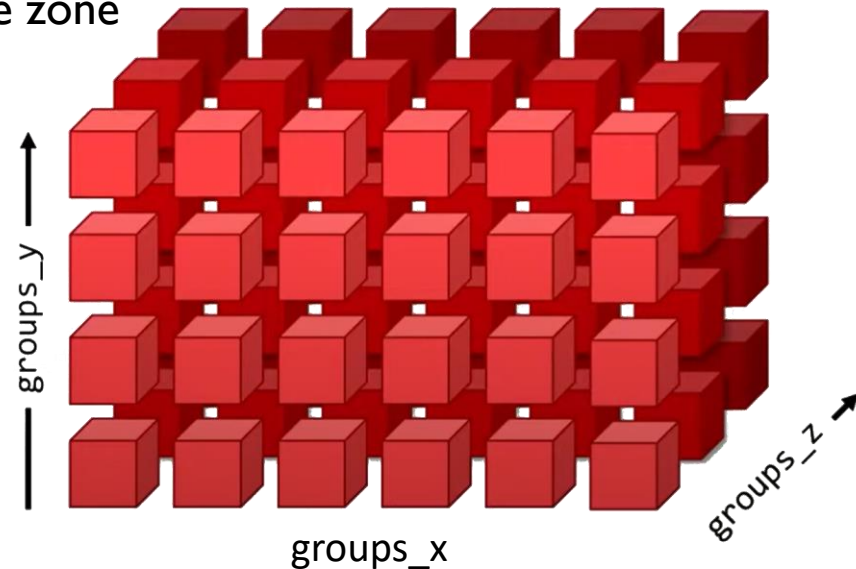
```
GLuint prog = glCreateProgram();  
glAttachShader( prog, cs );  
glLinkProgram( prog );
```


Dispatch a compute shader

```
glDispatchCompute( groups_x, groups_y, groups_z );
```

Number of
Work Groups

- Work groups :
 - Groupes de threads tournant sur une même zone (collection de cores)
 - Ont accès aux mêmes ressources locales
 - Peuvent être définis en 1D, 2D, 3D

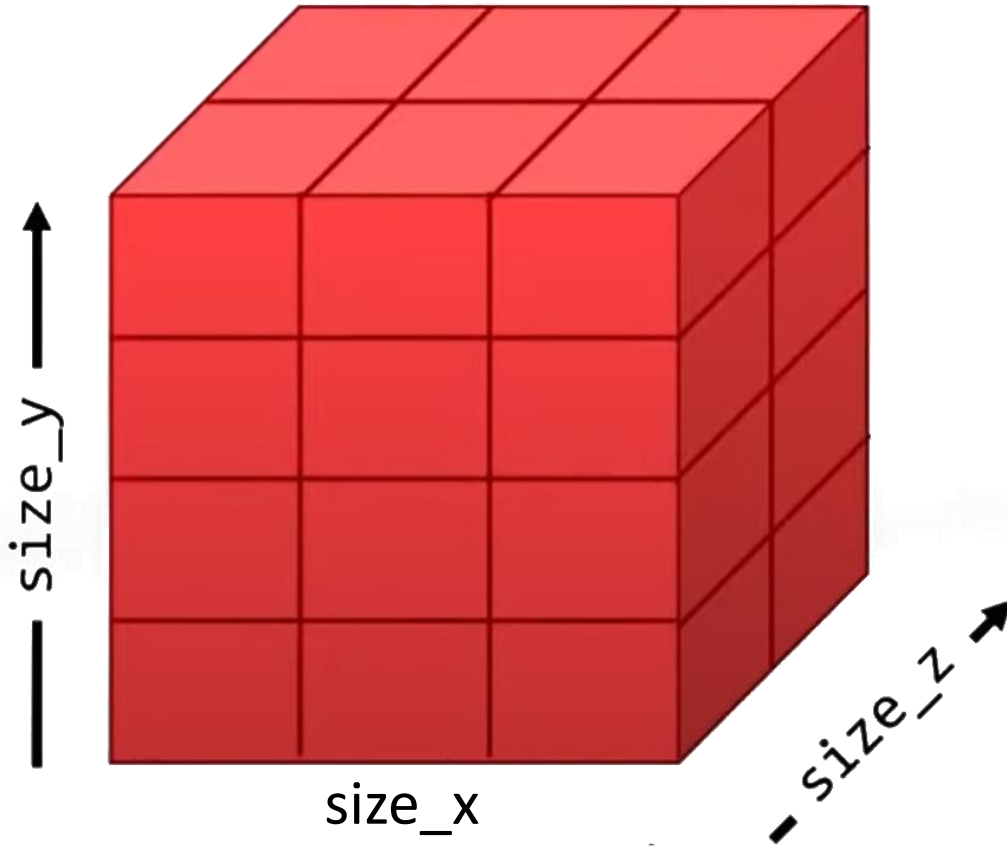


Work groups

- Taille d'un work group:
 - NVIDIA « wrap » de 32 coeurs
 - AMD « wavefront » de 64 coeurs
 - Optimale : 32 ou 64 (coeurs voisins)

```
layout ( local_size_x = 3,  
        local_size_y = 4,  
        local_size_z = 2 ) in;
```

GLSL



Compute Shaders

<code>uvec3 gl_NumWorkGroups</code>	Same as the <code>glDispatchCompute</code> call
<code>uvec3 gl_WorkGroupSize</code>	Same as layout <code>local_size_x/y/z</code>
<code>uvec3 gl_WorkGroupID</code>	Work group ID of this thread
<code>uvec3 gl_LocalInvocationID</code>	Invocation ID within this work group
<code>uvec3 gl_GlobalInvocationID</code>	Global invocation ID in all work groups
<code>uint gl_LocalInvocationIndex</code>	1D version of <code>gl_LocalInvocationID</code>

- Identifiants permettant de déterminer à quelles parties des données chaque exécution peut accéder (entrée/sortie)
 - Pas de limitations
 - Mais meilleurs performances si des threads voisins accèdent à des parties voisines

Example

GLSL :

```
#version 430 core

layout(std430, binding = 0) buffer myBuffer
{
    uvec3 data[];
} outBuffer;

layout ( local_size_x = 8, local_size_y = 4, local_size_z = 16 ) in;

void main()
{
    uvec3 gSize = gl_WorkGroupSize * gl_NumWorkGroups;
    uint  gPos  = gSize.x * gSize.y * gl_GlobalInvocationID.z
                + gSize.x * gl_GlobalInvocationID.y
                + gl_GlobalInvocationID.x;
    outBuffer.data[gPos] = gl_GlobalInvocationID;
}
```

Dispatch a Compute Shader

- Lancer l'exécution sur le GPU

```
glUseProgram( prog );  
glDispatchCompute( groups_x, groups_y, groups_z );
```

- Attendre que les calculs se terminent avant de faire les opérations suivantes :
 - Exemple : mouvement des particules CS puis rendu

```
// finish before using the output  
glMemoryBarrier( GL_SHADER_STORAGE_BARRIER_BIT );
```

→ Opérations qui utilisent la sortie

Example : Compute Shader



Alternatives au Compute Shaders



- Avantage du compute shader :
 - Code similaire aux autres shader (GLSL)
 - Le même contexte OpenGL → facilite l'utilisation directe de la sortie pour le rendu

Application au systèmes de particules

Particles Systems History

- Un des premiers modèles animés en CG

2/1
7



[Particle systems - A Technique for Modeling a
Class of Fuzzy Objects, William T. Reeves,
Lucasfilm, 1983 (Star Trek II)]



[Karl Sims, Particle dreams, 1988]

Example of particle system

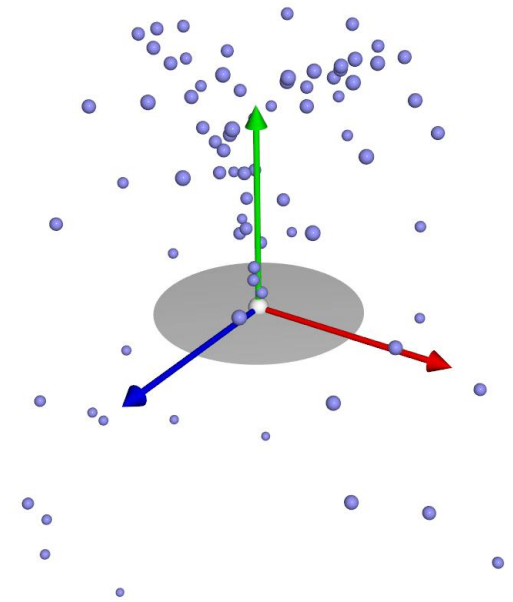
- **Spheres tombant avec la gravité**

- Representation géométrique de chaque particule : sphere

- Equation du mouvement $p(t) = \frac{1}{2}gt^2 + v_0t + p_0$

- Position et Vitesse peuvent être placés aléatoirement

- Chaque particule peut avoir sa propre durée de vie

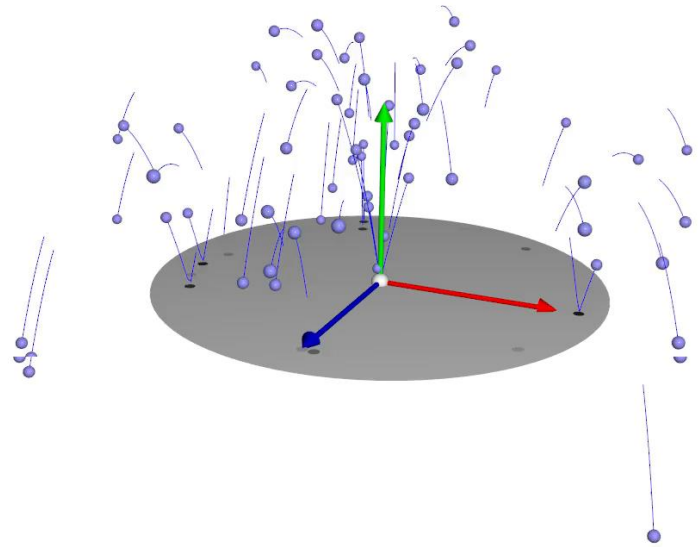


- *Q : Quels sont les paramètres d'entrée p_0 et v_0 ?*

$(x,y,z) = (\text{rouge}, \text{vert}, \text{bleu})$

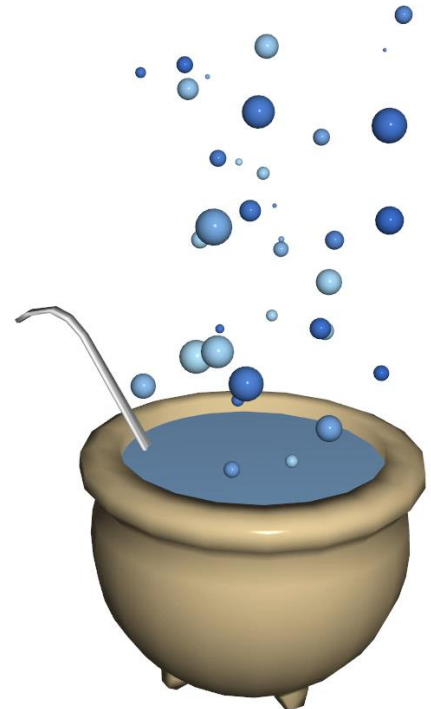
Bouncing spheres

- Q. What is the equation of motion ? (taking into account the bouncing) $p(t) = \dots$
- Help:
 - Consider a particle emitted at time $t = 0$
 - At what time t_i , the particle touches the ground ?
 - What is the new speed after impact ?
 - Express the complete equation of trajectory (piecewise definition)
- Bonus: How the "impact shadow" effect is computed ?



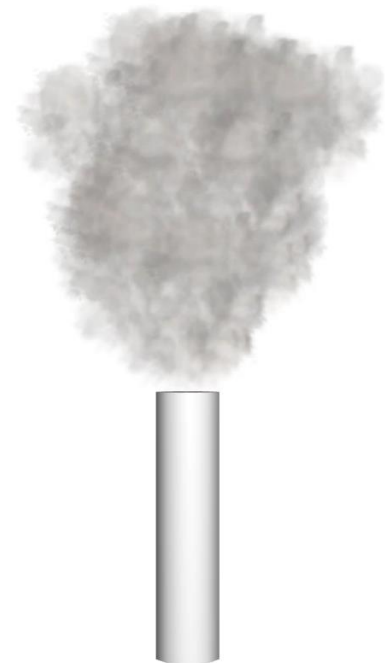
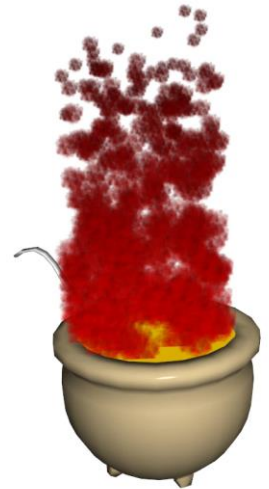
Manually defined/arbitrary motions

- Trajectories are not restricted to physics-based equations
- Q. *What are the parameters (and initial value) associated to each particle ?*
 - position = ...
 - radius = $\text{rand}([0,0.1])$
 - ...
- - *What is the equation of motion of a particle ?*
- - $p(t) = \dots$



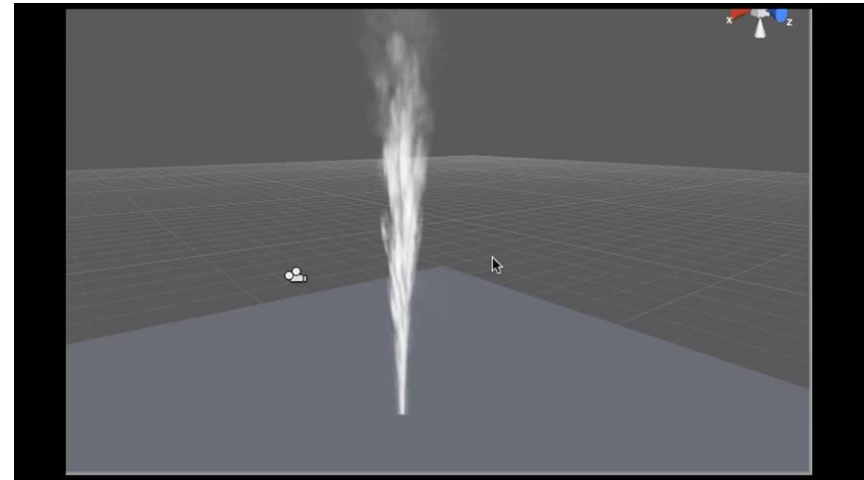
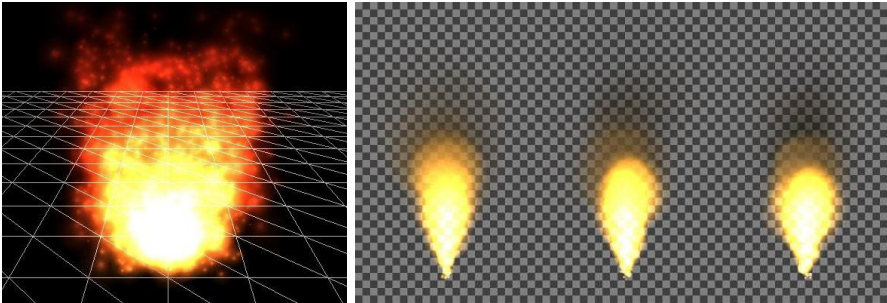
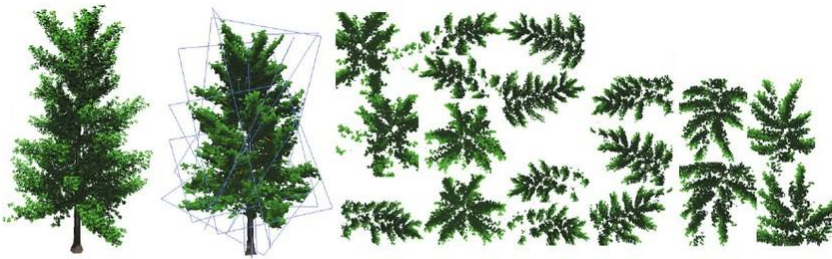
Billboards, Impostors, Sprites

- Particles can be displayed as small images/thumbnails (instead of simple sphere)
- In practice
 - Each particle is displayed as a quadrangle
 - A texture is mapped on the quad
- The texture can contain transparency : quad geometry is invisible
- Fake a more complex geometry
- The quad can be facing the camera at all time (billboard)
- The texture can be animated (sprite)
- Texture can be adapted to the point of view (impostors)



Usage of billboards

- Large use of billboards for complex models
 - *Vegetation, fire, smoke, etc.*



Example of billboard

Follow up of the example

Q. How to model the following animation ?



Use case in production

- How to model the *horse heads made of water* ?



The Lord of the Rings

Use case in production

- How to model the *horse heads made of water* ?



The Lord of the Rings

Use case in production

- Making off complet



The Lord of the Rings

Implementation - transparency in OpenGL

Transparency is not straightforward using projection/rasterization rendering.

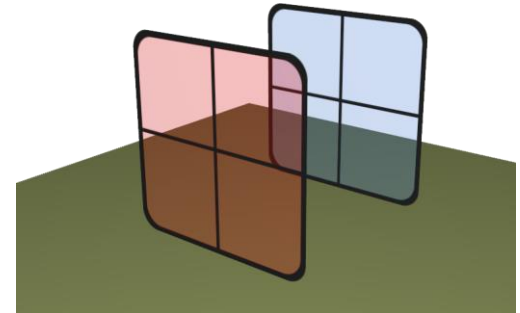
No perfect solution

⇒ Possibility to blend color for each pixel using α channel (rgba)

Need to be explicitly activated in OpenGL

```
glEnable(GL_BLEND);  
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);  
... Draw Calls ...  
glDisable(GL_BLEND);
```

- Various blending are possible for multiple effects (see [glBlendFunc](#))
- For transparency: (GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)
⇒ $\text{newColor} = \alpha \text{ currentColor} + (1 - \alpha) \text{ previousColor}$



Important note:

The **order** of the draw calls is important (previousColor/currentColor) The depth-buffer cannot handle the notion of transparency

Implementation - transparency in OpenGL

13/1
7

General Process:

- 1. Draw all **non-transparent** objects
- 2.a Activate color blending, deactivate depth buffer write
- 2.b **Sort** transparent objects by depth to the camera
$$p_{proj} = Proj \times ModelView \times p$$
$$z_{depth} = p_{proj} \cdot z / p_{proj} \cdot w$$
- 3. **Draw transparent objects** from furthest to closest

```
... Draw all non-transparent objects ...
glEnable(GL_BLEND); // Color Blending
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glDepthMask(false); // do not write on depth buffer
... Sort transparent objects by depth ...
... Draw all transparent objects from furthest to closest ...
glDisable(GL_BLEND);
// do not forget to re-activate depth buffer write
glDepthMask(true);
```

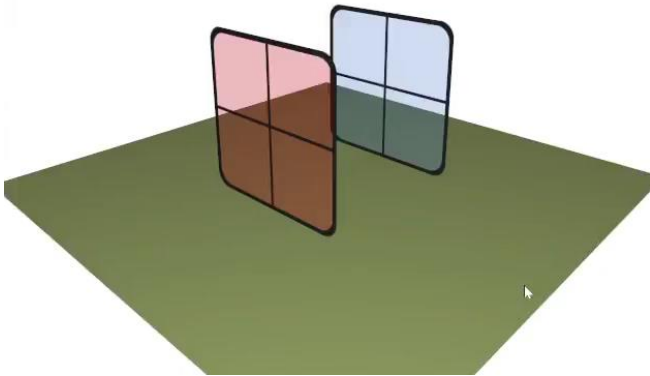
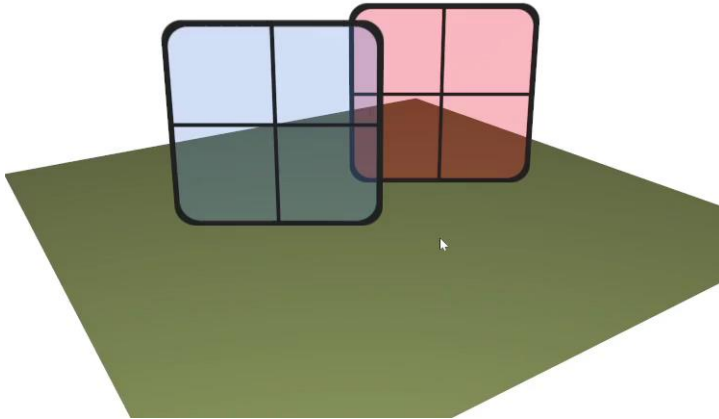


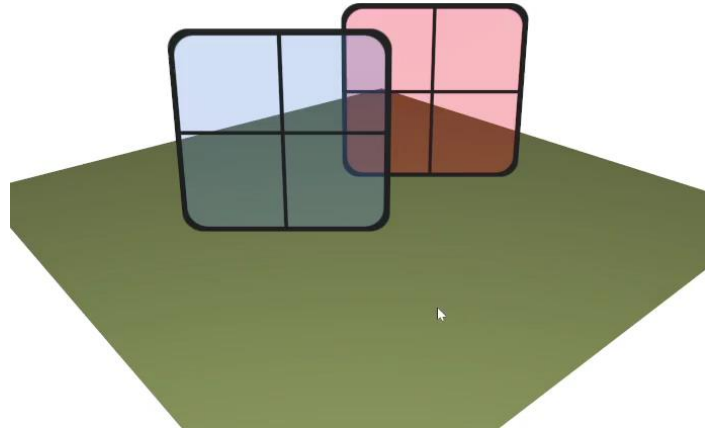
Image Blending - common errors

Keeping depth-buffer write activated
(without sorting)



Strong artifact, cannot be used

No sorting, depth buffer write de-activated



Note:

Can be OK for blurry sprites (like smokes) where
element order is hard to distinguish.
Avoids the cost of the sorting.

Implementation - transparency in OpenGL

- For impostors shape with fully opaque/transparent fragment:
 - **discard** can be used on transparent fragments
 - *discard = Stop fragment shader execution (nothing is written)*
 - (+) Generic, standard display
 - *no need to sort shapes, depth-buffer works*
 - (-) Possible aliasing artifacts
 - (-) Cannot be used on semi-transparent objects



In fragment shader:

```
float limit = 0.6;  
if( texture.a < limit) {  
    discard;  
}
```

Instancing

In particle system we need to display lots of similar shapes

Multiple draw calls ($N > 1000$) can be costly

each draw call requires synchronization b/w CPU-GPU

```
// Slow for large N
for(int k=0; k<N; ++k)
    someOperator(k)
    draw(shape) // ... glDrawElements(...)
```

Solution: **Use Instancing**

glDrawArrays → [glDrawArraysInstanced](#)

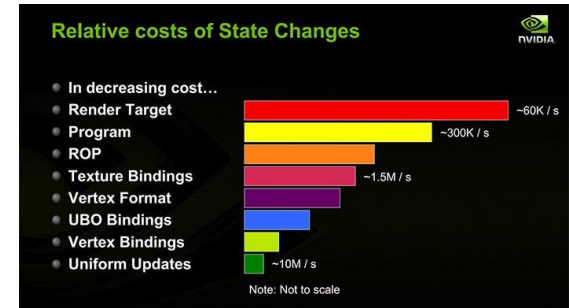
glDrawElements → [glDrawElementsInstanced](#)

In the C++ code

```
// 1 unique call, N is a parameter
drawInstanced(shape, N) // ... glDrawElementsInstanced(...,N)
// No more explicit loop
```

In the Shader:

```
// ...
someOperator(gl_InstanceID)
// ...
```



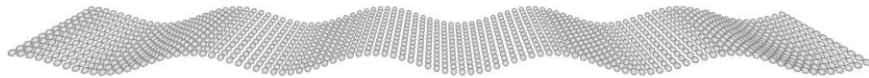
[[NVIDIA](#)]

(+) Full capability of your GPU

(-) Require a specific shader

Instancing Demo

Without Instancing



Slow for 15,000 particles
#vertices=750,000

With Instancing



Can display 450,000 particles at 60fps (depends on GPU)
#vertices=22,000,000

En TP

- Utilisation des nouveaux shaders pour la synthèse de particules
 - Mouvement calculé avec le compute shader
 - Rendu avec le geometry shader
- Utilisation de billboard
- Calcul de mouvement physiques et procéduraux

Fin